COMSATS UNIVERSITY ISLAMABAD

ATTOCK CAMPUS

LAB 5
(Tasks)

SUBJECT

Data Structure

Submitted By : Inza Bibi

Reg No : FA25-BCS-031



Lab Task 1

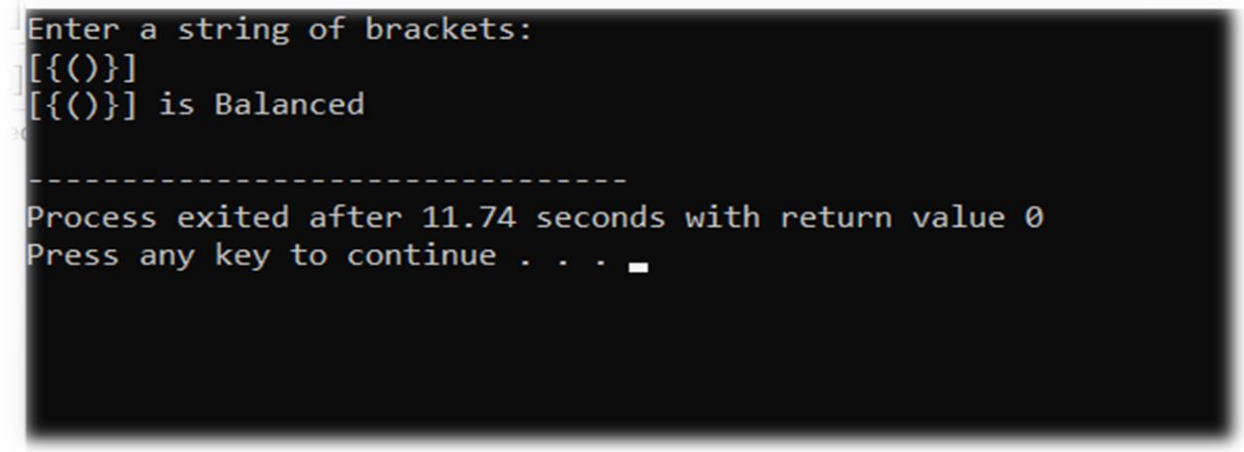
Write an application using Stack that checks whether the entered string of brackets is balanced, e.g. [{( )}] is Balanced while {[( )] } is not Balanced because the priority of the pranthesis is not maintained.

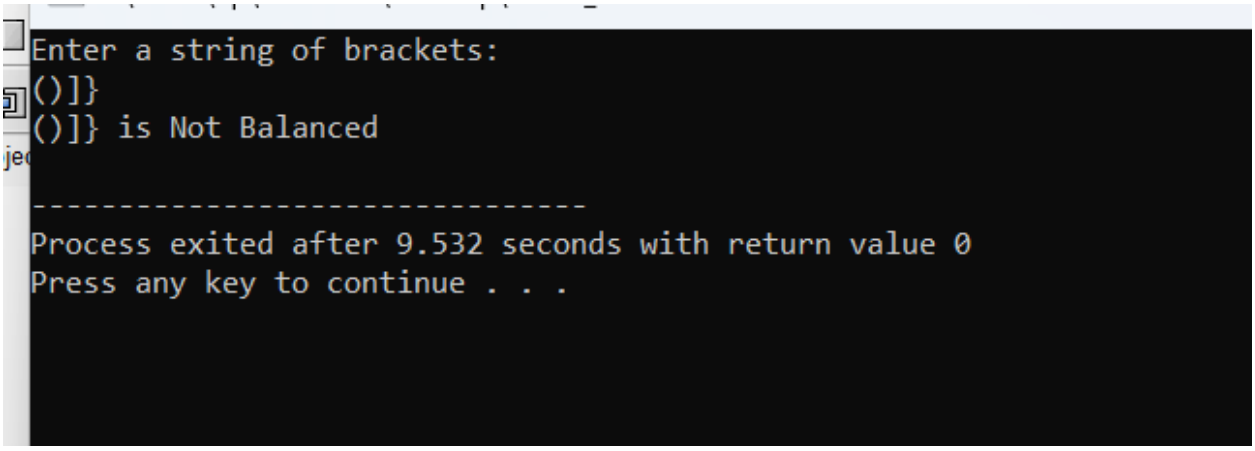
```
#include <iostream>
#include <string>
using namespace std;
const int MAX = 100;
// Stack structure
struct Stack {
    char data[MAX];     // Array to store brackets
    int top;            // Index of top element
};
// Initialize stack
void initStack(Stack &s) {
    s.top = -1;         // -1 means empty
}
// Check if stack is empty
bool isEmpty(Stack &s) {
    return (s.top == -1);
}
// Push (add element)
void push(Stack &s, char value) {
    if (s.top == MAX - 1) {
        cout << "Stack Overflow!" << endl;
        return;
```

```
    }
    s.top++;
    s.data[s.top] = value;
}
// Pop (remove element)
char pop(Stack &s) {
    if (isEmpty(s)) {
        return '\0';    // Return null character if empty
    }
    char value = s.data[s.top];
    s.top--;
    return value;
}
// Peek (view top element)
char peek(Stack &s) {
    if (isEmpty(s)) {
        return '\0';
    }
    return s.data[s.top];
}
// Check if brackets match
bool isMatching(char opening, char closing) {
    if (opening == '(' && closing == ')') return true;
    if (opening == '{' && closing == '}') return true;
    if (opening == '[' && closing == ']') return true;
    return false;
```

```
}
// Check if brackets are balanced
bool isBalanced(string str) {
   Stack s;
   initStack(s);
   // Traverse the string
   for (int i = 0; i < str.length(); i++) {
      char ch = str[i];
      // If opening bracket, push to stack
      if (ch == '(' || ch == '{' || ch == '[') {
         push(s, ch);
      }
      // If closing bracket
      else if (ch == ')' || ch == '}' || ch == ']') {
         // If stack is empty, no matching opening bracket
         if (isEmpty(s)) {
            return false;
         }
         // Pop the top and check if it matches
         char opening = pop(s);
         if (!isMatching(opening, ch)) {
            return false; } } }
   // After traversal, stack should be empty
   return isEmpty(s);
}
int main() {
```

```
string str;
cout << "Enter a string of brackets: ";
cin >> str;
if (isBalanced(str)) {
    cout << str << " is Balanced" << endl;
} else {
    cout << str << " is Not Balanced" << endl;
}
return 0;}
```





Lab Task 2:

Use dynamic stack and implement Infix to Postfix conversion algorithm and test it for various inputs.

```
#include <iostream>
#include <string>
using namespace std;
// ==================== DYNAMIC STACK ====================
// Node structure for stack
struct Node {
    char data;      // Stores the character (operator or bracket)
    Node* next;     // Points to the next node
};
Node* top = NULL;   // Top of the stack, initially NULL
// Check if stack is empty
bool isEmpty() {
    return (top == NULL);
}
```

```
// Push an element onto the stack
void push(char value) {
   Node* newNode = new Node;   // Create a new node
   newNode->data = value;      // Store the value
   newNode->next = top;        // New node points to old top
   top = newNode;              // Update top to new node
}
// Pop an element from the stack
char pop() {
   if (isEmpty()) {
      return '\0';    // Return null if empty
   }
   char value = top->data;     // Get top value
   Node* temp = top;           // Store top node
   top = top->next;            // Move top down
   delete temp;                // Free memory
   return value;               // Return popped value
}
// Peek at the top element without removing
char peek() {
   if (isEmpty()) {
      return '\0';
   }
```

```
    return top->data;
}

// ==================== HELPER FUNCTIONS ====================

// Check if character is an operator
bool isOperator(char ch) {
    return (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '^');
}
// Check if character is an operand (letter or digit)
bool isOperand(char ch) {
    return ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z') || (ch >= '0' && ch <=
'9'));
}
// Get priority of an operator
int getPriority(char ch) {
    if (ch == '^') return 3;
    if (ch == '*' || ch == '/') return 2;
    if (ch == '+' || ch == '-') return 1;
    return 0;
}

// ==================== INFIX TO POSTFIX ====================
```

```
string infixToPostfix(string infix) {
    string postfix = "";    // Result string
    // Traverse each character of infix expression
    for (int i = 0; i < infix.length(); i++) {
        char ch = infix[i];
        // Case 1: Operand → Add to output
        if (isOperand(ch)) {
            postfix += ch;
        }
        // Case 2: Opening bracket → Push to stack
        else if (ch == '(') {
            push(ch);
        }
        // Case 3: Closing bracket → Pop until '('
        else if (ch == ')') {
            while (!isEmpty() && peek() != '(') {
                postfix += pop();   // Add popped operators to output
            }
            if (!isEmpty()) pop();  // Remove '(' from stack
        }
        // Case 4: Operator
        else if (isOperator(ch)) {
```

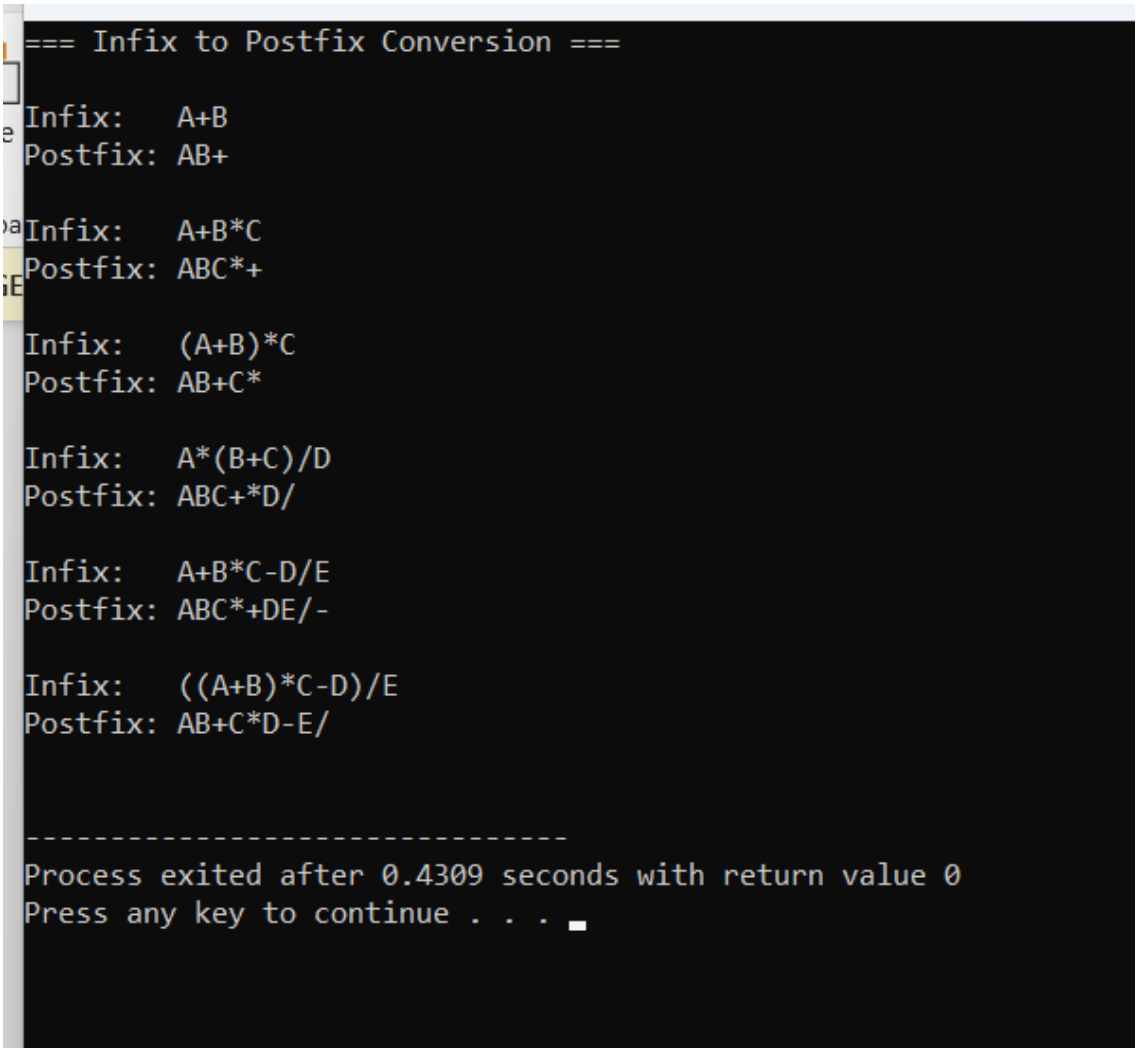
```
            // Pop operators with higher or equal priority
            while (!isEmpty() && getPriority(peek()) >= getPriority(ch)) {
                postfix += pop();
            }
            push(ch);    } } // Push current operator
    // Pop all remaining operators
    while (!isEmpty()) {
        postfix += pop();
    }
    return postfix;
}

// ==================== MAIN ====================

int main() {
    // Test various inputs
    string inputs[] = {
        "A+B",
        "A+B*C",
        "(A+B)*C",
        "A*(B+C)/D",
        "A+B*C-D/E",
        "((A+B)*C-D)/E"
```

```
};
cout << "=== Infix to Postfix Conversion ===" << endl << endl;
for (int i = 0; i < 6; i++) {
   // Reset stack for each input
   while (!isEmpty()) pop();
   string result = infixToPostfix(inputs[i]);
   cout << "Infix:   " << inputs[i] << endl;
   cout << "Postfix: " << result << endl << endl;
}
return 0;
```

}


=== Infix to Postfix Conversion ===
Infix: A+B
Postfix: AB+
Infix: A+B*C
Postfix: ABC*+
Infix: (A+B)*C
Postfix: AB+C*
Infix: A*(B+C)/D
Postfix: ABC+*D/
Infix: A+B*C-D/E
Postfix: ABC*+DE/-
Infix: ((A+B)*C-D)/E
Postfix: AB+C*D-E/
Process exited after 0.4309 seconds with return value 0
Press any key to continue . . .

Task 03:

For a given postfix expression, use dynamic stack to evaluate a numerical result for given values of variables.

```
#include <iostream>
using namespace std;
// ==================== DYNAMIC STACK ====================
struct Node {
    int data;
    Node* next;
};
Node* top = NULL;
bool isEmpty() {
    return (top == NULL);
}
void push(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = top;
    top = newNode;
}
int pop() {
    if (isEmpty()) {
        cout << "Stack Underflow!" << endl;
```

```
        return -1;
    }
    int value = top->data;
    Node* temp = top;
    top = top->next;
    delete temp;
    return value;
}
// ==================== MANUAL CHARACTER CHECKS
====================
// Check if character is an uppercase letter (A-Z)
bool isUpperCase(char ch) {
    return (ch >= 'A' && ch <= 'Z');
}

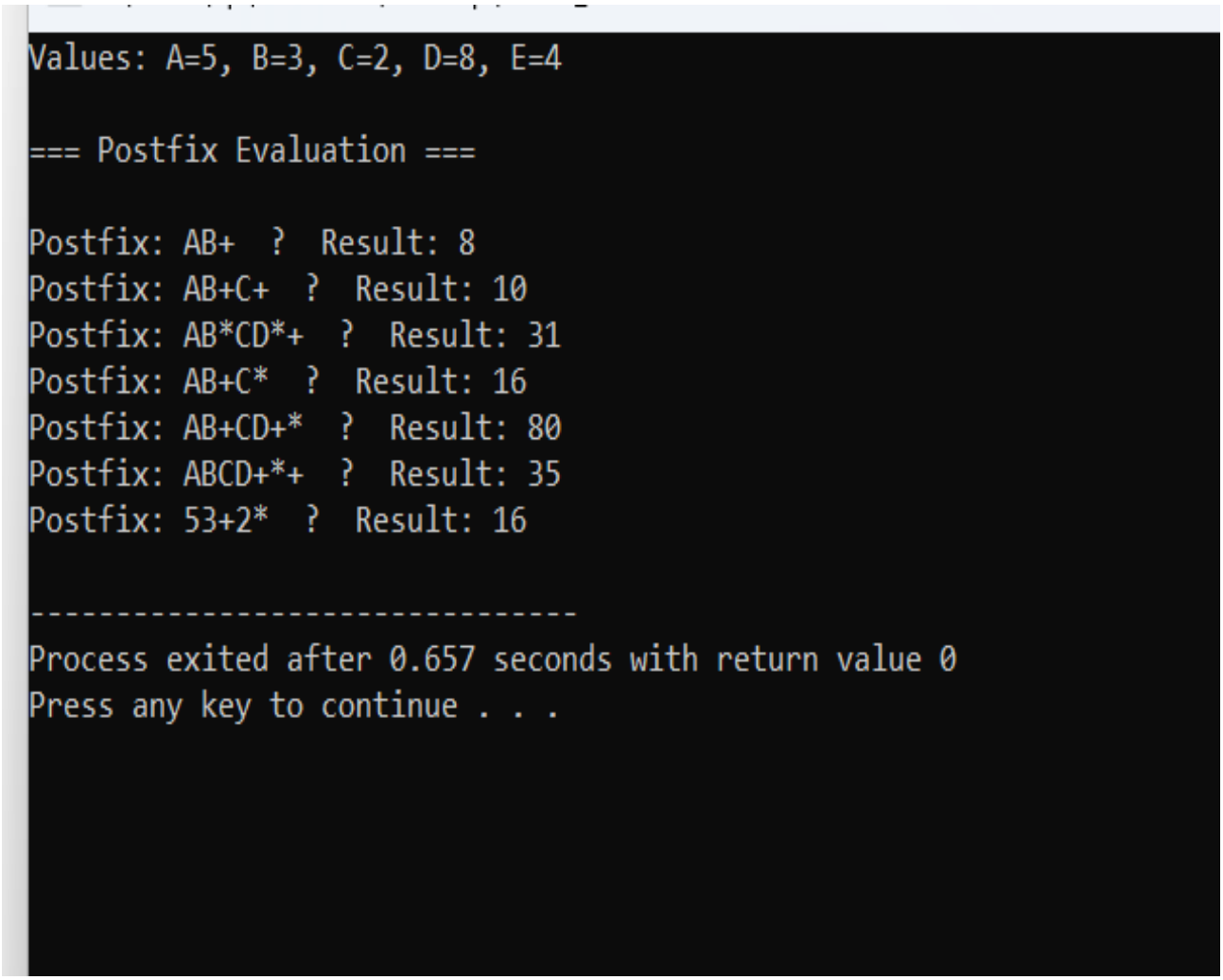
// Check if character is a lowercase letter (a-z)
bool isLowerCase(char ch) {
    return (ch >= 'a' && ch <= 'z');
}
// Check if character is a letter (A-Z or a-z)
bool isLetter(char ch) {
    return (isUpperCase(ch) || isLowerCase(ch));
}
```

```
// Check if character is a digit (0-9)
bool isDigit(char ch) {
   return (ch >= '0' && ch <= '9');
}
// Check if character is an operator
bool isOperator(char ch) {
   return (ch == '+' || ch == '-' || ch == '*' || ch == '/');
}
// ==================== APPLY OPERATION ====================
int applyOperation(int a, int b, char op) {
   if (op == '+') return a + b;
   if (op == '-') return a - b;
   if (op == '*') return a * b;
   if (op == '/') return a / b;
   return 0;
}
// ==================== POSTFIX EVALUATION ====================
int evaluatePostfix(string postfix, int values[]) {
   // Clear stack
   while (!isEmpty()) pop();
   // Traverse each character
   for (int i = 0; i < postfix.length(); i++) {
      char ch = postfix[i];
```

```
        // Skip spaces
        if (ch == ' ') continue;
        // Case 1: Uppercase letter (A-Z) → Push its value
        if (isUpperCase(ch)) {
            int index = ch - 'A';       // A=0, B=1, C=2...
            push(values[index]);
        }
        // Case 2: Digit (0-9) → Push its numeric value
        else if (isDigit(ch)) {
            push(ch - '0');             // Convert char to int
        }
        // Case 3: Operator → Pop 2, apply, push result
        else if (isOperator(ch)) {
            int second = pop();
            int first = pop();
            int result = applyOperation(first, second, ch);
            push(result);
        }
    }
    return pop();
}

// ==================== MAIN ====================
```

```
int main() {
    int values[26] = {0};
    values['A' - 'A'] = 5;
    values['B' - 'A'] = 3;
    values['C' - 'A'] = 2;
    values['D' - 'A'] = 8;
    values['E' - 'A'] = 4;
    cout << "Values: A=5, B=3, C=2, D=8, E=4" << endl << endl;
    string expressions[] = {
        "AB+",
        "AB+C+",
        "AB*CD*+",
        "AB+C*",
        "AB+CD+*",
        "ABCD+*+",
        "53+2*"
    };
    cout << "=== Postfix Evaluation ===" << endl << endl;
    for (int i = 0; i < 7; i++) {
        int result = evaluatePostfix(expressions[i], values);
        cout << "Postfix: " << expressions[i]
            << "  →  Result: " << result << endl; }
     return 0; }
```


Values: A=5, B=3, C=2, D=8, E=4
=== Postfix Evaluation ===
Postfix: AB+ ? Result: 8
Postfix: AB+C+ ? Result: 10
Postfix: AB*CD*+ ? Result: 31
Postfix: AB+C* ? Result: 16
Postfix: AB+CD+* ? Result: 80
Postfix: ABCD+*+ ? Result: 35
Postfix: 53+2* ? Result: 16
Process exited after 0.657 seconds with return value 0
Press any key to continue . . .